

Lecture 2: Introduction to R Markdown and R Basics

Heather Mattie, PhD

September 3, 2026

Recipe of the Day!

Apple cheddar soup



Announcements

- No lab this week
- There will be lab next week
- No TF office hours this week
- Heather's office hour this week is today 2-2:30pm

Office Hours and Lab

Office Hours

Day	Time	Staff	Location
Thursday	12-1 pm	Heather	Building 1, 4 th floor, room 421A
Wednesday	10:30-11:30 am	Stephanie	Building 2, 4 th floor, room 426
Friday	11:30-12:30 pm	Monica	Building 2, 4 th floor, room 426
Monday	1-2 pm	Kimberly	Zoom

Lab

Day	Time	Location
Friday	9:45-11:15 am	Zoom

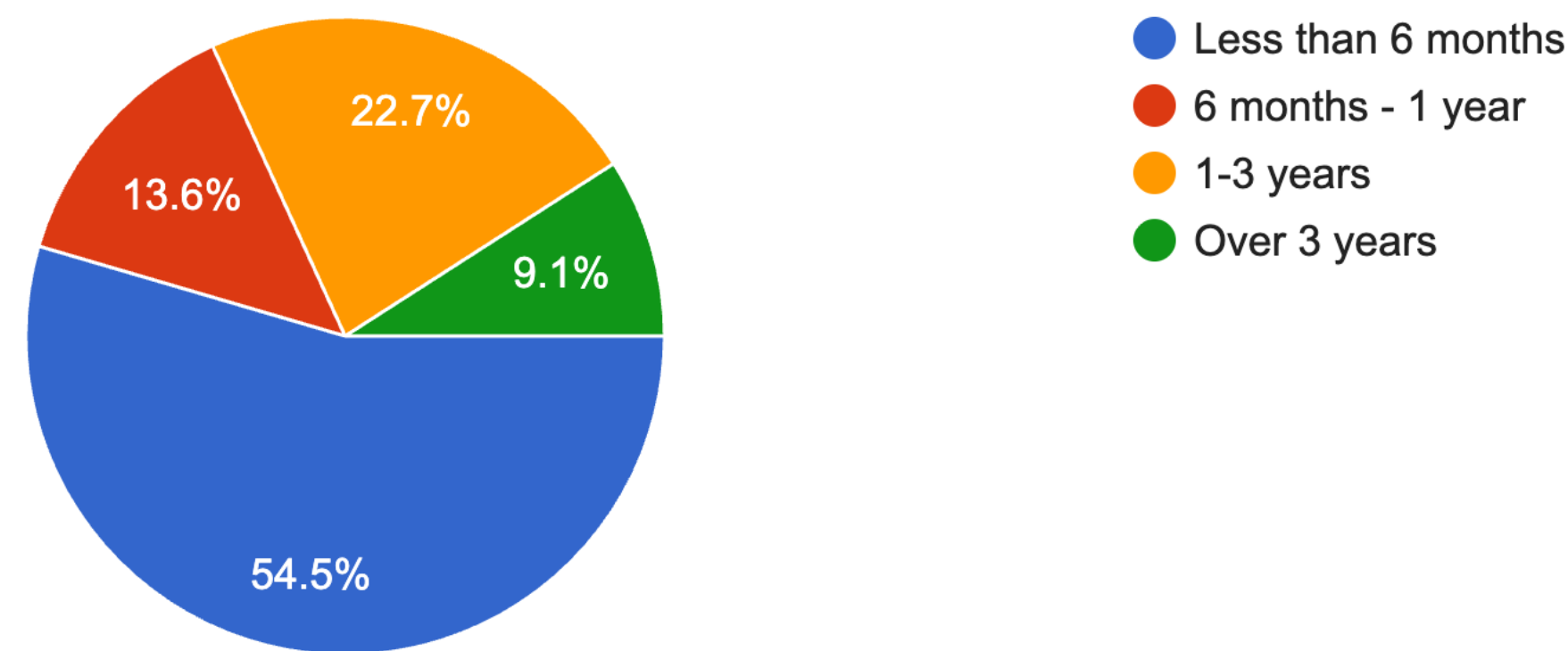
***Lab will not be held every week**



Survey results

How long have you been programming?

22 responses



Q2 **MULTIPLE CHOICE** 43 responses

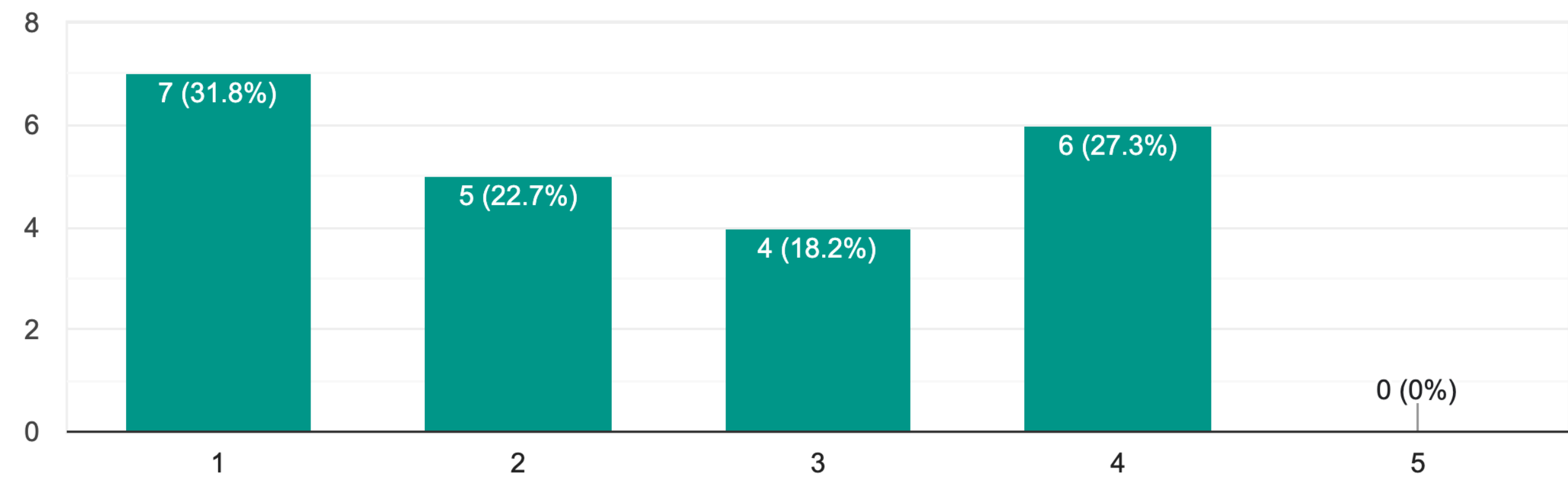
How long have you been coding?



Survey results

Overall, how comfortable are you with programming?

22 responses



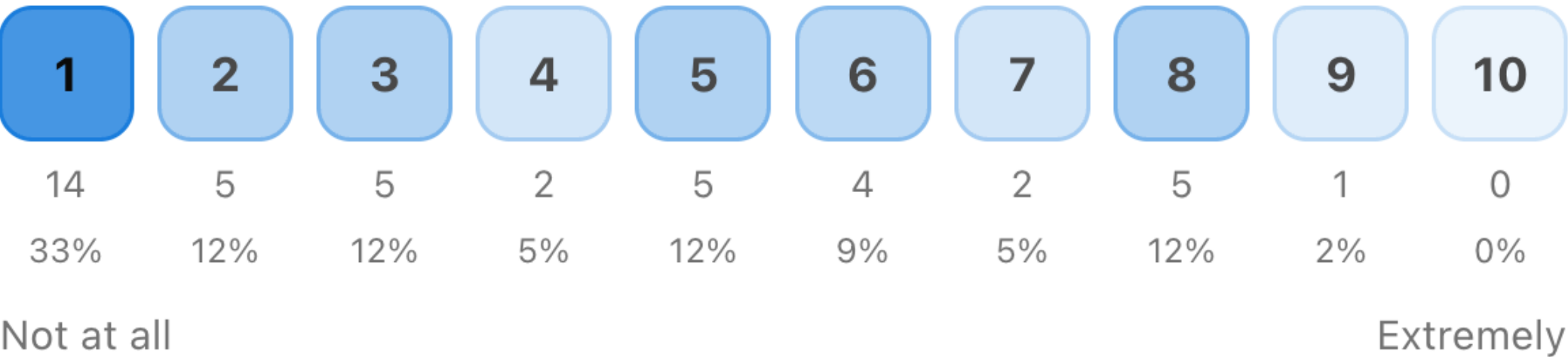
Q3

NUMERIC SCALE

43 responses

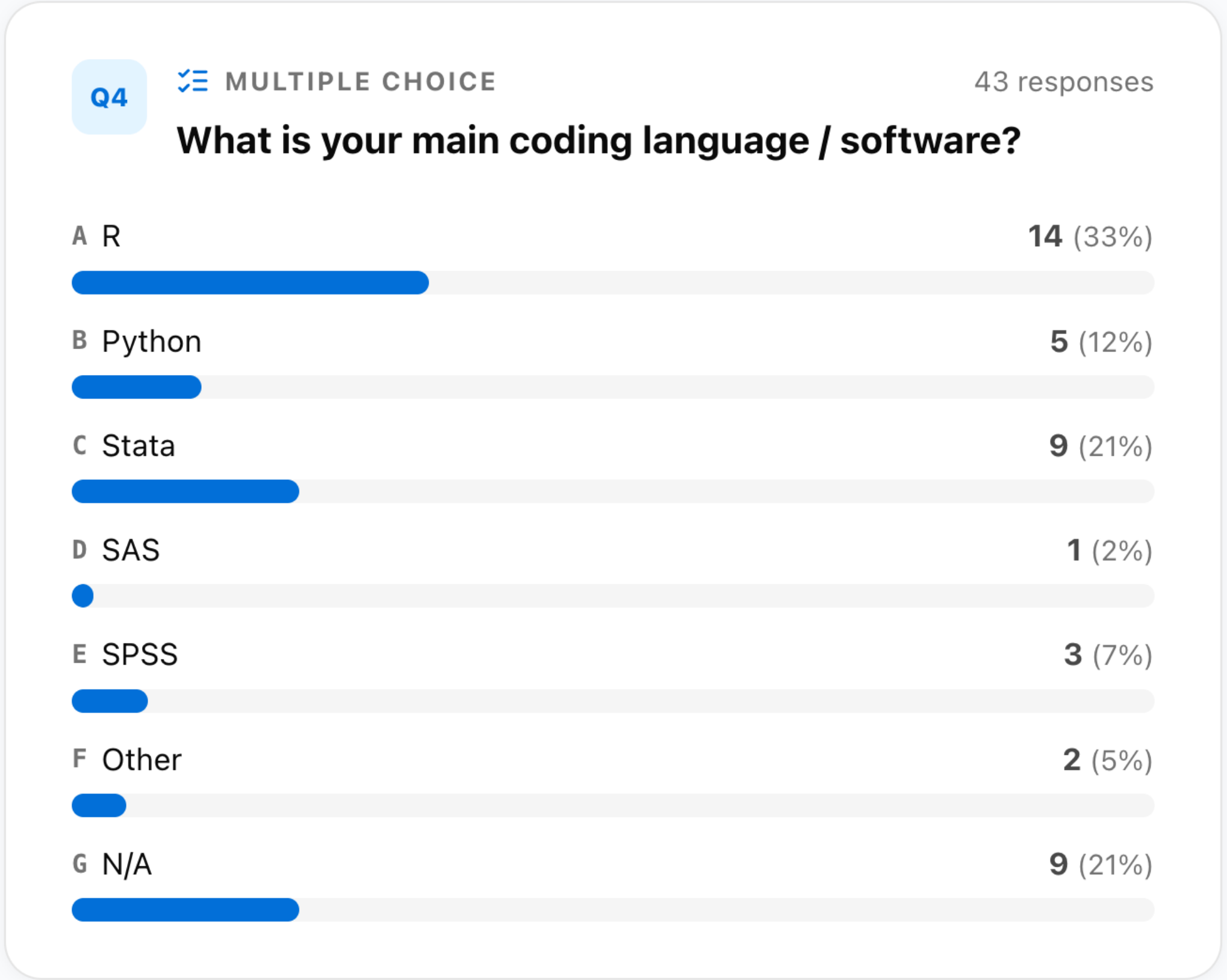
Overall, how comfortable are you with coding?

3.7 avg score



Survey results

- **R proficiency:** A clear majority of respondents identify R as their primary programming language.
- **Stata usage:** Several individuals utilize Stata, with some noting a transition from Stata to R or using both languages for their coursework and research.
- **Additional languages:** A smaller subset of respondents uses Python or possesses a background in C++, while some participants report no prior programming experience or limited exposure through introductory courses.



Survey results

What do you hope to get out of this course?

- **R Programming Proficiency:** A primary goal is to achieve comfort and independence in using R for data cleaning, manipulation, visualization, and efficient processing to support research and clinical analysis.
- **Data Science & Machine Learning:** Participants are seeking to understand statistical methods and acquire specific skills in machine learning to facilitate the analysis of large, complex datasets.
- **Professional Development:** Learners aim to align their coding capabilities with job market demands and acquire formal programming techniques to conduct independent public health and medical research.

To leave this course being able to know when a claim is being made carelessly

Be able to take a data set and use coding to produce informative graphics, statistical analyses, describe trends

Ability to better analyze and visualize program and outcomes data to inform large policy decisions

To get baseline understanding of coding and machine learning. My aim is to create ai scribe software in future.

Learn R, I also work with Claude Code so to be better at understanding code will help me at my job.

To be proficient enough in R to code for clinical research projects (i.e. chart reviews using EMR, making pretty graphs, implementing stat tests like t-tests, p-values on data set)

Introduction to R, RStudio and R Markdown

Introduction to R Markdown

Heather Mattie, PhD

What is R Markdown?

- **R Markdown** is a tool for creating dynamic documents that **combine** text, code, and the output of that code into a **single** file



SOURCE EDITOR

1. New File

5. Save and Render

6. Share

4. Set Output Format(s) and Options

3. Write Text

2. Embed Code

```
1 ---
2 title: "Document Title"
3 author: "Author Name"
4 output:
5   html_document:
6     toc: TRUE
7 ---
8
9 ```{r setup, include=FALSE}
10 knitr::opts_chunk$set(echo = TRUE)
11 ```
12
13 ## R Markdown
14
15 This is an R Markdown document.
16 Markdown is a simple formatting
17 syntax for authoring HTML, PDF,
18 and MS Word documents.
19
20 ```{r cars}
21 summary(cars)
22 ```
```

set preview location

insert code chunk

go to code chunk

run code chunk(s)

show outline

run all previous chunks

modify chunk options

run current chunk

Console Terminal R Markdown Jobs

~/Desktop/report/report.Rmd

Output created: report.html

RENDERED OUTPUT

file path to output document

report.html Open in Browser Find Publish

find in document

Document Title

Author Name

- R Markdown
- Including Plots

R Markdown

reload document

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

summary(cars)

##	speed	dist
##	Min. : 4.0	Min. : 2.00
##	1st Qu.: 12.0	1st Qu.: 26.00
##	Median : 15.0	Median : 36.00
##	Mean : 15.4	Mean : 42.98
##	3rd Qu.: 19.0	3rd Qu.: 56.00
##	Max. : 25.0	Max. : 120.00

report.html 802.9 KB Jul 9, 2021, 4:35 PM

What is Markdown?

Markdown is a lightweight markup language that uses simple text formatting to structure and style text.

- Plain text formatting
- Easy formatting
- Readability
- Versatile

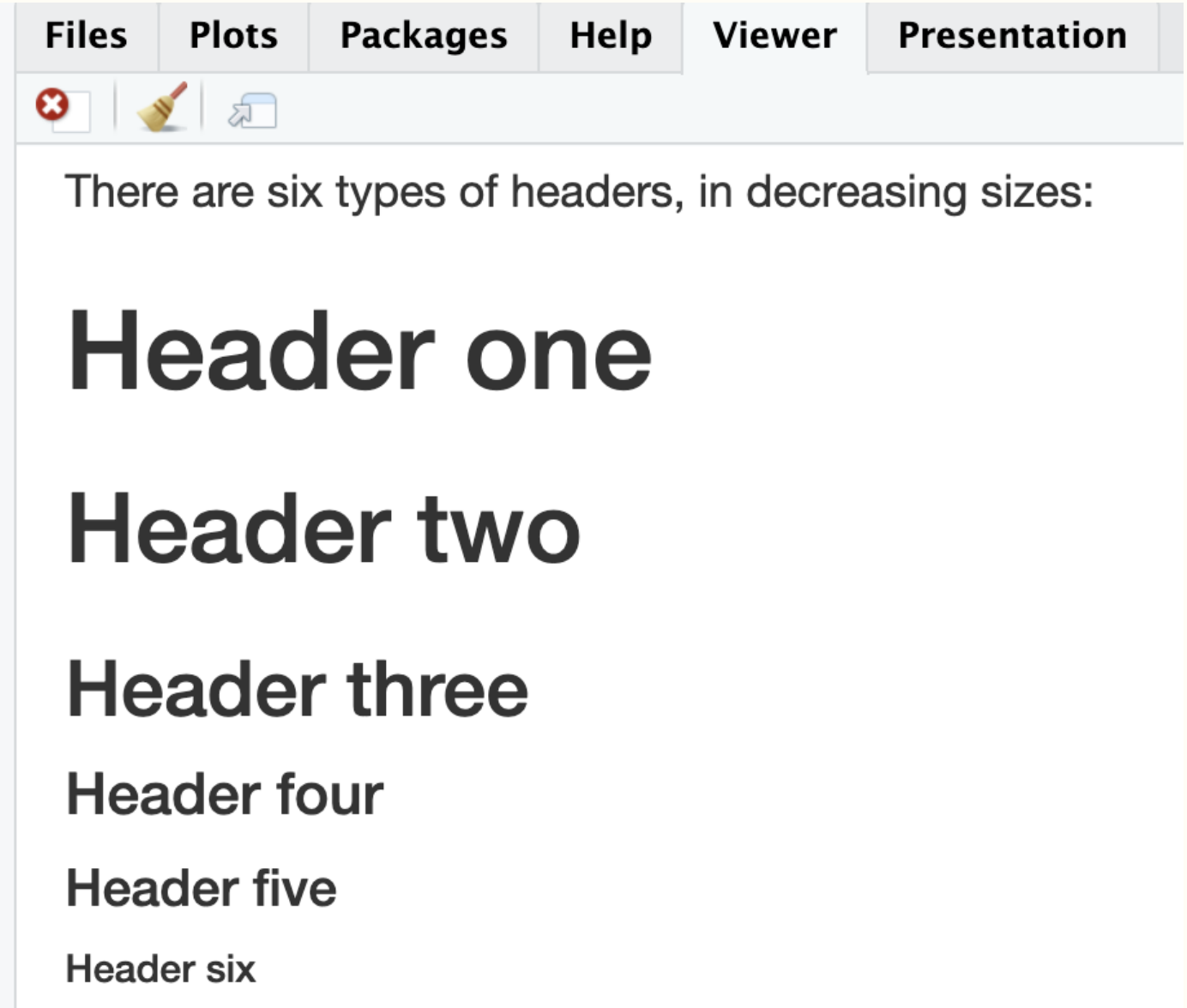


Headings

To denote a **heading**, add a **number sign** (hashtag) before it

- The more number signs, the smaller the text

```
43
44
45
46 There are six types of headers, in decreasing
    sizes:
47
48 # Header one
49
50 ## Header two
51
52 ### Header three
53
54 #### Header four
55
56 ##### Header five
57
58 ##### Header six
59
60
```



Bold and Italics

To make a word or string of words *italic*, surround the word or words with 1 asterisk (*) or 1 underscore (_)

To make a word or string of words **bold**, surround the word or words with 2 asterisks (**) or 2 underscores (__)

```
29 ### Italics and Bold
30
31 _This_ is one word in italics.
32
33 _This is a string of words in italics._
34
35 **This** is one word in bold.
36
37 **This is a string of words in bold.**
38
39 _**This is a few words**_ in **bold** and
   _italics_.
```

Italics and Bold

This is one word in italics.

This is a string of words in italics.

This is one word in bold.

This is a string of words in bold.

This is a few words in **bold** and *italics*.

Lists

You can create **unordered** and **ordered** lists in Markdown.

- To create an **unordered** list, preface each item with an asterisk (*) and give each list item its own line.
- To create an **ordered** list, preface each item with a number and give each item its own line.

139 To create an unordered list, you'll want to preface each item in the
140 list with an asterisk (`*``). Each list item also gets its own line. For
141 example, a grocery list in Markdown might look like this:

```
141 * Milk  
142 * Eggs  
143 * Salmon  
144 * Butter
```

145
146 An ordered list is prefaced with numbers, instead of asterisks. For
example:

```
147  
148 1. Crack three eggs over a bowl  
149 2. Pour a gallon of milk into the bowl  
150 3. Rub the salmon vigorously with butter  
151 4. Drop the salmon into the egg-milk bowl
```

To create an unordered list, you'll want to preface each item in the list with an asterisk (*). Each list item also gets its own line. For example, a grocery list in Markdown might look like this:

- Milk
- Eggs
- Salmon
- Butter

An ordered list is prefaced with numbers, instead of asterisks. For example:

1. Crack three eggs over a bowl
2. Pour a gallon of milk into the bowl
3. Rub the salmon vigorously with butter
4. Drop the salmon into the egg-milk bowl

Hyperlinks

There are two link types in Markdown:

1. To create an **inline link**, wrap the link text in brackets ([]) and the link in parentheses ().
2. A **reference link** is used when you want to reference the same link multiple times in your document. A specific name is selected as the reference, put in square brackets, and then a colon and the link are placed next to it. To use the reference link, wrap the link text in brackets and the reference link name in brackets.

```
90 ▾ ### Links
91
92 ▾ #### Inline links
93
94 Visit [this website](https://rmarkdown.rstudio.com/lesson-1
    .html) for more details about R Markdown!
95
96
97 ▾ #### Reference links
98
99 Here's [a link to something else][another place].
100 Here's [yet another link][another link].
101 And now back to [the first link][another place].
102
103 [another place]: https://rstudio.github.io/cheatsheets/
104 [another link]: https://www.hsph.harvard.edu/
```

Links

Inline links

Visit [this website](https://rmarkdown.rstudio.com/lesson-1.html) for more details about R Markdown!

Reference links

Here's [a link to something else](#).

Here's [yet another link](#).

And now back to [the first link](#).

Images

There are two image types in Markdown:

1. To create an **inline image**, use the exact same syntax as an inline hyperlink, but with an exclamation point (!) in the beginning.
2. A **reference image** also uses nearly the same syntax as a reference hyperlink, but with an exclamation point (!) in the beginning.

Images

There are two image types in Markdown:

1. To create an **inline image**, use the exact same syntax as an inline hyperlink, but with an exclamation point (!) in the beginning.
2. A **reference image** also uses nearly the same syntax as a reference hyperlink, but with an exclamation point (!) in the beginning.

```
125 ▾ ### Images
126
127 ▾ #### Inline images
128
129 For example, to create an inline image link to
130 https://octodex.github.com/images/bannekat.png, with an alt text
131 that says, Benjamin Bannekat, you'd write this in Markdown:
132
133 !\[Benjamin Bannekat\](https://octodex.github.com/images/bannekat.png)
134 { width=25% }
135
```

Images

Inline images

For example, to create an inline image link to <https://octodex.github.com/images/bannekat.png>, with an alt text that says, Benjamin Bannekat, you'd write this in Markdown:



Benjamin Bannekat

Images

There are two image types in Markdown:

1. To create an **inline image**, use the exact same syntax as an inline hyperlink, but with an exclamation point (!) in the beginning.
2. A **reference image** also uses nearly the same syntax as a reference hyperlink, but with an exclamation point (!) in the beginning.

```
136 - #### Reference images
137
138 ![The first father][First Father]
139
140 ![The second first father][Second Father]
141
142 [First Father]: http://octodex.github.com/images/founding-father.jpg { width=50% }
143
144 [Second Father]:http://octodex.github.com/images/foundingfather\_v2.png { width=50% }
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
```

Reference images



The first father

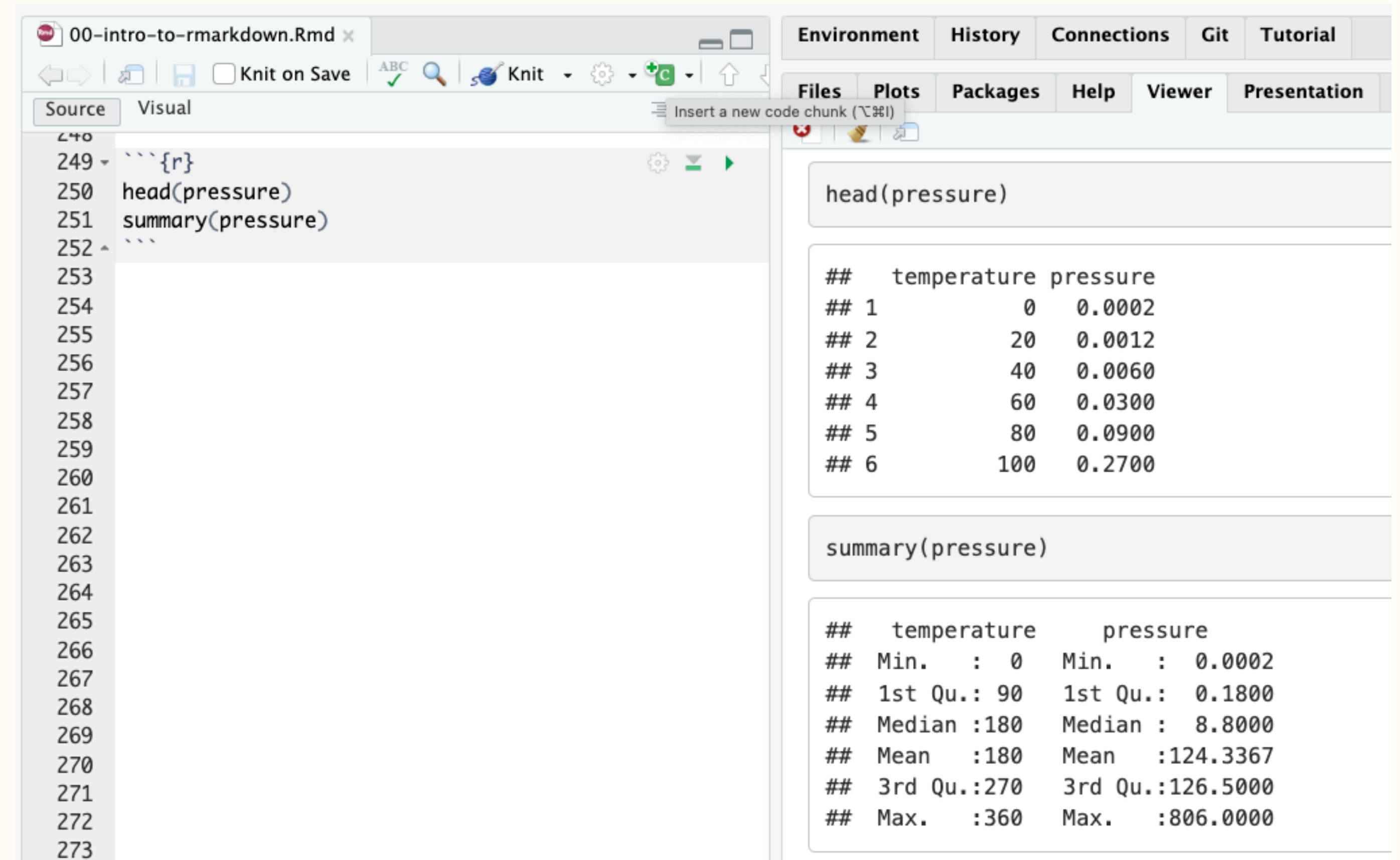


The second first father

Embedding code

There are three ways to create a code chunk:

1. Typing ````${r}```` to start a code chunk and then ````` to end it
2. Clicking the **Add Chunk** button in the tool bar
3. The keyboard shortcut
Ctrl + Alt + I
(Mac: **Cmd + Option + I**)



The screenshot shows the RStudio interface with a file named "00-intro-to-rmarkdown.Rmd". The "Source" pane on the left shows a code chunk starting at line 249 with the code:

```
249 ```{r}
250 head(pressure)
251 summary(pressure)
252 ```
```

The "Environment" pane on the right shows the output of the code chunk. It displays the results of `head(pressure)` and `summary(pressure)`.

head(pressure)

##	temperature	pressure
## 1	0	0.0002
## 2	20	0.0012
## 3	40	0.0060
## 4	60	0.0300
## 5	80	0.0900
## 6	100	0.2700

summary(pressure)

##	temperature	pressure
## Min. :	0	Min. : 0.0002
## 1st Qu.: 90		1st Qu.: 0.1800
## Median :180		Median : 8.8000
## Mean :180		Mean :124.3367
## 3rd Qu.:270		3rd Qu.:126.5000
## Max. :360		Max. :806.0000

Embedding code

There are different display options for code and code output

- Adding a comma and **echo = FALSE** after the **r** displays the code output but not the code itself
- Adding a comma and **include = FALSE** prevents code and output from appearing in the finished file

```
249 ```{r, echo = FALSE}
250 head(pressure)
251 summary(pressure)
252 ```
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
```

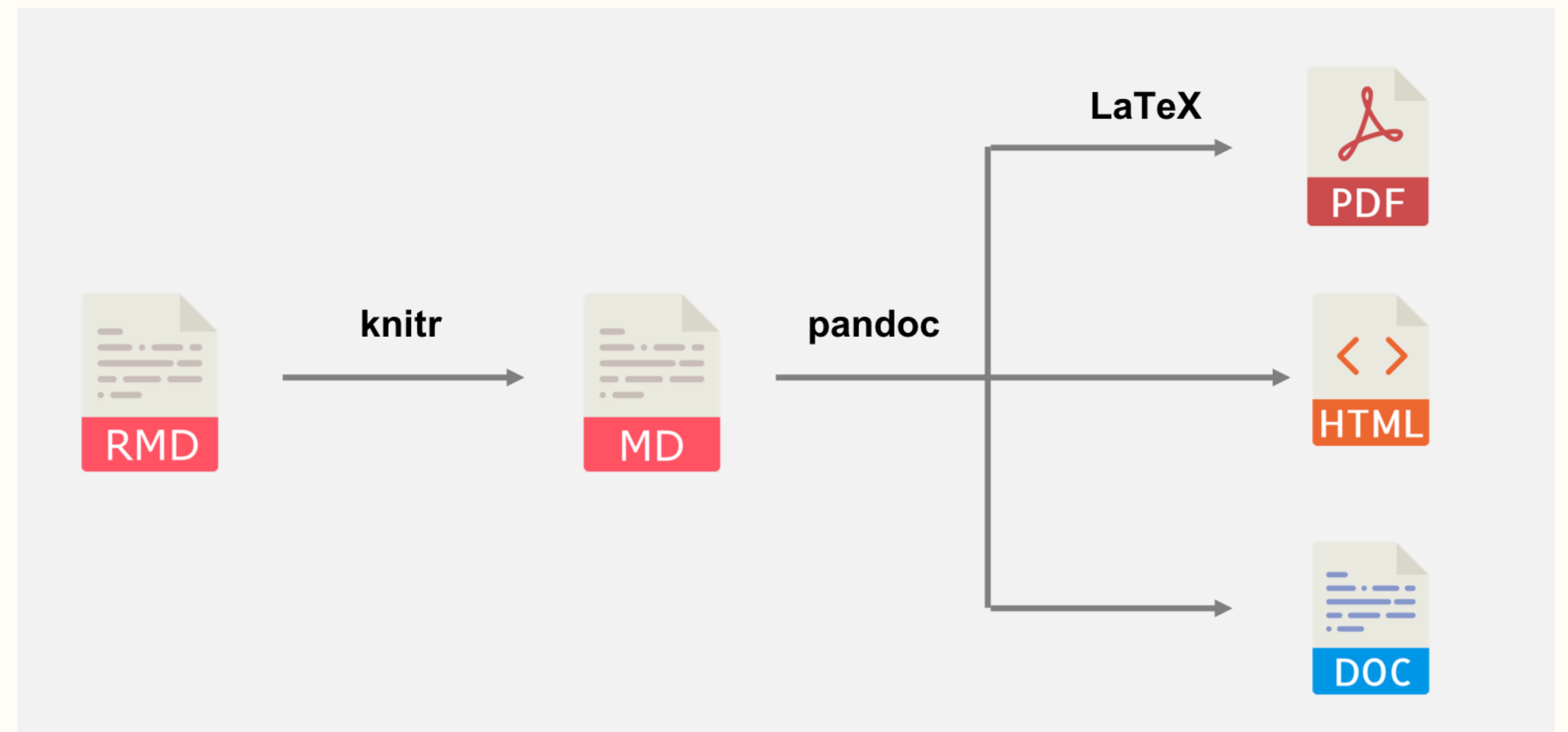
##	temperature	pressure
## 1	0	0.0002
## 2	20	0.0012
## 3	40	0.0060
## 4	60	0.0300
## 5	80	0.0900
## 6	100	0.2700

##	temperature	pressure
## Min. :	0	Min. : 0.0002
## 1st Qu.:	90	1st Qu.: 0.1800
## Median :	180	Median : 8.8000
## Mean :	180	Mean :124.3367
## 3rd Qu.:	270	3rd Qu.:126.5000
## Max. :	360	Max. :806.0000

Compiling R Markdown files

How does an R Markdown file (.Rmd file) compile?

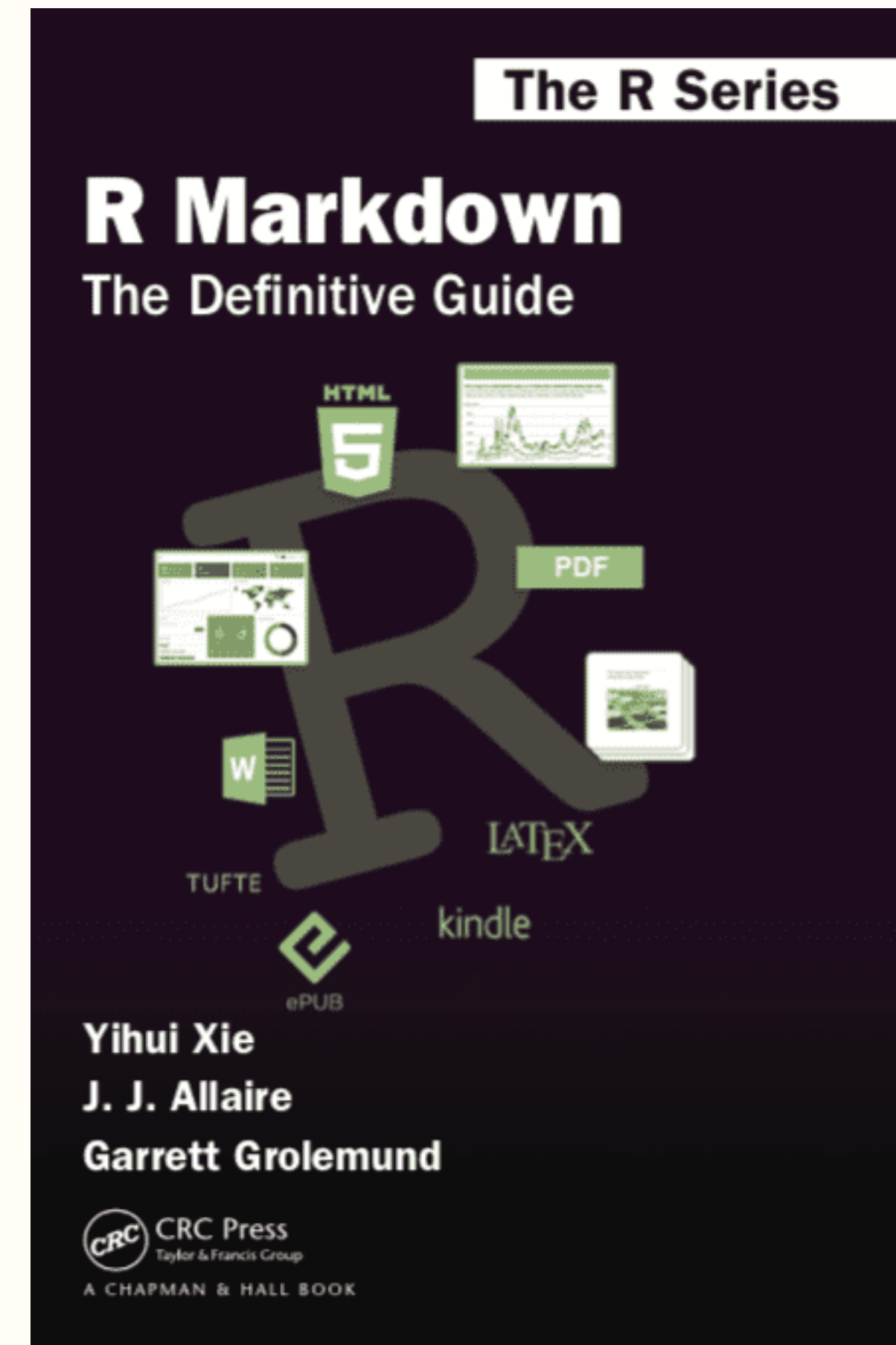
- The **rmarkdown** package renders an R Markdown file by executing other packages that knit the text and code chunks together into an output file
- The output file can be a Word doc, PDF, or HTML file
- The file will not knit (compile) if there is a bug in the code



Other document types

R Markdown can also be used to produce

- PDF presentations with beamer
- HTML presentations with ioslides and slidy
- Full academic manuscripts
- Dashboards
- Websites
- Interactive documents



Summary

- R Markdown is a tool that can create documents, slides, websites, etc. that combine text and code into one file
- Markdown is the text formatting markup language used in R Markdown files

R Basics

R Basics Roadmap

1. Data types
2. Vectors
3. Sorting
4. Indexing
5. Basic plots
6. Programming basics

R Basics

Data Types

Heather Mattie, PhD

Data types

A **data type** is a classification that specifies the kind of value a variable can hold and how R should interpret and handle it

- Numeric (integer or decimal)
- Text (string, character)
- True/False value (Boolean, logical)
- Data frame
- List

Data types

Integer

```
x <- 2  
class(x)
```

```
## [1] "numeric"
```

Decimal

```
y <- 3.14  
class(y)
```

```
## [1] "numeric"
```

Text

```
z <- "Boston"  
class(z)
```

```
## [1] "character"
```

Logical

```
a <- TRUE  
class(a)
```

```
## [1] "logical"
```

Data frame

```
library(dslabs)  
data(gapminder)  
d <- gapminder  
class(d)
```

```
## [1] "data.frame"
```

List

```
record <- list(name = "John Doe",  
               student_id = 1234,  
               grades = c(95, 82, 91, 97, 93),  
               final_grade = "A")  
class(record)
```

```
## [1] "list"
```

Examining objects

The function **str** displays more about the structure of an object

```
library(dslabs)
data(gapminder)
str(gapminder)
```

```
## 'data.frame':   10545 obs. of  9 variables:
## $ country      : Factor w/ 185 levels "Albania","Algeria",...: 1 2 3 4 5 6 7 8 9 10 ...
## $ year         : int  1960 1960 1960 1960 1960 1960 1960 1960 1960 1960 ...
## $ infant_mortality: num  115.4 148.2 208 NA 59.9 ...
## $ life_expectancy : num  62.9 47.5 36 63 65.4 ...
## $ fertility     : num  6.19 7.65 7.32 4.43 3.11 4.55 4.82 3.45 2.7 5.57 ...
## $ population    : num  1636054 11124892 5270844 54681 20619075 ...
## $ gdp           : num  NA 1.38e+10 NA NA 1.08e+11 ...
## $ continent     : Factor w/ 5 levels "Africa","Americas",...: 4 1 1 2 2 3 2 5 4 3 ...
## $ region        : Factor w/ 22 levels "Australia and New Zealand",...: 19 11 10 2 15 21 2 1 22 21 ...
```

Examining objects

The function **head** displays the first 6 rows and all columns of a data frame

```
head(gapminder)
```

```
##           country year infant_mortality life_expectancy fertility
## 1      Albania 1960      115.40           62.87           6.19
## 2      Algeria 1960      148.20           47.50           7.65
## 3       Angola 1960      208.00           35.98           7.32
## 4 Antigua and Barbuda 1960      NA           62.97           4.43
## 5      Argentina 1960      59.87           65.39           3.11
## 6      Armenia 1960      NA           66.86           4.55
## population      gdp continent      region
## 1    1636054      NA    Europe Southern Europe
## 2    11124892 13828152297    Africa Northern Africa
## 3     5270844      NA    Africa  Middle Africa
## 4      54681      NA  Americas    Caribbean
## 5    20619075 108322326649  Americas  South America
## 6     1867396      NA     Asia  Western Asia
```

The accessor

The **accessor operator \$** is used to access different variables.

Most commonly, it is used to access the values in a column in a data frame.

```
library(dslabs)
data(admissions)
names(admissions)
```

```
## [1] "major"      "gender"     "admitted"   "applicants"
```

```
admissions$major
```

```
## [1] "A" "B" "C" "D" "E" "F" "A" "B" "C" "D" "E" "F"
```

Vectors

The `admissions$major` object is not one character, but several – it is a **vector**.

- The number of entries in a vector can be displayed with the **length** function

```
majors <- admissions$major  
length(majors)
```

```
## [1] 12
```

```
class(majors)
```

```
## [1] "character"
```

Factors

Factors are a data type useful for storing categorical data.

R stores these *levels* (categories) as integers but keeps track of the labels by keeping a map of the integer to the level. This is more memory efficient than storing all the characters.

```
head(gapminder)
```

```
##           country year infant_mortality life_expectancy fertility
## 1         Albania 1960          115.40           62.87         6.19
## 2         Algeria 1960          148.20           47.50         7.65
## 3          Angola 1960          208.00           35.98         7.32
## 4 Antigua and Barbuda 1960             NA           62.97         4.43
## 5         Argentina 1960           59.87           65.39         3.11
## 6          Armenia 1960             NA           66.86         4.55
## population      gdp continent      region
## 1    1636054      NA    Europe Southern Europe
## 2    11124892 13828152297    Africa Northern Africa
## 3     5270844      NA    Africa Middle Africa
## 4      54681      NA Americas Caribbean
## 5    20619075 108322326649 Americas South America
## 6    1867396      NA     Asia Western Asia
```

```
class(gapminder$country)
```

```
## [1] "factor"
```

Summary

- A data type specifies the kind of value a variable can be and how the R should interpret and handle it
- The **str** function displays the structure of an object
- The **head** function displays the first 6 rows of a data frame
- The accessor **\$** is used to access different variables

R Basics

Vectors

Heather Mattie, PhD

Creating vectors

Vectors are created using the concatenate function **c**.

Vectors with text or characters as entries must have quotation marks around each entry.

- Without quotation marks, R will look for variables with those names
- An error will be produced

```
codes <- c(380, 124, 818)
codes
```

```
## [1] 380 124 818
```

```
country <- c("italy", "canada", "egypt")
country
```

```
## [1] "italy" "canada" "egypt"
```

```
country <- c(italy, canada, egypt)
```

Names

It might be useful to **name** the entries of a vector

- Connect the entry with a name for the entry with an equal sign
- Names can be coded with or without quotation marks
- Names can also be assigned using the **names** function

```
codes <- c(italy = 380, canada = 124, egypt = 818)
codes
```

```
## italy canada egypt
##    380    124    818
```

```
names(codes)
```

```
## [1] "italy" "canada" "egypt"
```

```
codes <- c("italy" = 380, "canada" = 124, "egypt" = 818)
codes
```

```
## italy canada egypt
##    380    124    818
```

```
codes <- c(380, 124, 818)
country <- c("italy", "canada", "egypt")
names(codes) <- country
codes
```

```
## italy canada egypt
##    380    124    818
```

Sequences

The **seq** function creates vector sequences

- The first argument defines the start
- The second argument defines the end
- The default increment is to increase by 1 but this can be customized
- There is shorthand for consecutive numbers
- The default class of the entries in a sequence is integer but will be numeric if decimal numbers are part of the sequence

```
seq(1, 10)
```

```
## [1] 1 2 3 4 5 6 7 8 9 10
```

```
seq(1, 10, 2)
```

```
## [1] 1 3 5 7 9
```

```
1:10
```

```
## [1] 1 2 3 4 5 6 7 8 9 10
```

```
class(1:10)
```

```
## [1] "integer"
```

```
class(seq(1, 10, 0.5))
```

```
## [1] "numeric"
```

Subsetting

Square brackets are used to access specific elements in a vector

- The position of the entry in the vector, or the name of the entry can be used to access it
- A vector or sequence of numbers can also be used inside the square brackets

```
codes
```

```
## italy canada egypt  
## 380 124 818
```

```
codes[2]
```

```
## canada  
## 124
```

```
codes[c(1,3)]
```

```
## italy egypt  
## 380 818
```

```
codes[1:2]
```

```
## italy canada  
## 380 124
```

```
codes["canada"]
```

```
## canada  
## 124
```

```
codes[c("egypt", "italy")]
```

```
## egypt italy  
## 818 380
```

Coercion

Coercion is R attempting to be flexible with data types

- The entries in a vector must be the same data type
- If an entry isn't the type expected, R will guess what it is and **coerce** the value to be a certain type if it can
- Otherwise, it will give an error
- Numeric entries **can** be coerced into characters
- Character entries **cannot** be coerced into numbers

```
x <- c(1, "canada", 3)
x
```

```
## [1] "1"      "canada" "3"
```

```
class(x)
```

```
## [1] "character"
```

```
x <- 1:5
y <- as.character(x)
y
```

```
## [1] "1" "2" "3" "4" "5"
```

```
as.numeric(y)
```

```
## [1] 1 2 3 4 5
```

Not Availables (NA)

When a coercion function encounters an impossible case, it gives a warning and turns the entry into an **NA**

- NA stands for not available
- NA is also used to denote missing data

```
x <- c("1", "b", "3")  
as.numeric(x)
```

```
## Warning: NAs introduced by coercion
```

```
## [1] 1 NA 3
```

Vector arithmetic

Vector arithmetic involves performing mathematical operations on vectors

- Addition, subtraction, multiplication, division, etc.
- In R, arithmetic operations on vectors occur *element wise*

```
heights <- c(69, 62, 66, 70, 70, 73, 67, 73, 67, 70)
```

```
heights * 2.54
```

```
## [1] 175.26 157.48 167.64 177.80 177.80 185.42 170.18 185.42 170.18 177.80
```

```
heights - 69
```

```
## [1] 0 -7 -3 1 1 4 -2 4 -2 1
```

Two vectors

If we have two vectors of the same length, and we sum them, they are added entry by entry

- The same holds for subtraction, multiplication, and division

$$\begin{pmatrix} a \\ b \\ c \\ d \end{pmatrix} + \begin{pmatrix} e \\ f \\ g \\ h \end{pmatrix} = \begin{pmatrix} a + e \\ b + f \\ c + g \\ d + h \end{pmatrix}$$

Summary

- Vectors are objects with multiple entries that can be created with the concatenate function
- All entries in a vector must be of the same data type
- The entries in a vector can be named
- The **seq** function produces a sequence vector
- Square brackets are used to subset a vector
- R will use coercion if the data types of entries in a vector conflict